

Ticket 003 Detection Engineering Report

Ticket #003 - Detection Engineering Report

Executive Summary

This investigation looked at a practical SOC question: when do repeated SSH login failures stop looking like ordinary user error and start looking like activity worth investigating?

I first established a normal baseline on the Ubuntu lab host. That baseline included a successful SSH login and a simple user-error case in which one incorrect password was followed by a successful login from the normal Windows source. I then generated a controlled suspicious sequence from the Kali lab VM: five failed passwords against the same account in 67 seconds, with no successful login afterward.

Version 1 of the detector grouped failures by username and source IP and applied a five-failures-in-120-seconds threshold. It correctly alerted on the Kali activity. A separate legitimate-user test, however, produced the same count from the normal source and then logged in successfully. Version 1 alerted on that sequence as well, which exposed a false-positive problem.

Version 2 kept the original threshold but added source familiarity and successful-login context. In the lab, that change suppressed the legitimate retry scenario while preserving the alert on the unusual Kali source.

The final result is a working proof of concept that shows why authentication detections benefit from sequence and source context instead of relying on failure counts alone.

Detection Objective

The detector is intended to identify repeated failed SSH password attempts against the same Linux account from the same source IP over a short period, then use surrounding authentication context to help separate likely password guessing from routine user mistakes.

The high-priority path should stay quiet for isolated failures and should avoid escalating the specific lab scenario in which repeated failures come from an established source and are followed shortly by a successful login.

Environment

All work was performed in an authorized cybersecurity lab.

Monitored Linux Host

- Hostname: `ubuntu-soc-lab`
- Operating system: Ubuntu 24.04.4 LTS
- Primary lab IP: `192.168.56.121`
- Authentication service: OpenSSH

- Authentication log: `/var/log/auth.log`
- Test account: `soc-test`
- Test account UID: `1003`

Normal Authentication Source

- System: Windows host
- Source IP: `192.168.56.1`
- Role in testing: normal login baseline and legitimate-retry testing

Suspicious Test Source

- System: Kali Linux lab VM
- Source IP: `192.168.56.111`
- Role in testing: controlled suspicious authentication activity

Detector

- Language: Python 3
- Version 1: `detection/scripts/ssh_auth_detection_v1.py`
- Version 2: `detection/scripts/ssh_auth_detection_v2.py`
- Current version: `detection/scripts/ssh_auth_detection.py`

Normal Behavior / Baseline

I established the baseline before generating the suspicious sequence so the detector would have a clear reference for ordinary SSH behavior.

Successful Login

The `soc-test` account successfully authenticated from the normal Windows source at `192.168.56.1`. The log showed a successful password event, the SSH session opening, normal session activity, and the session closing.

Evidence:

- `testing/baseline/02-normal-soc-test-login.log`

Benign Password Mistake

I then simulated a common user mistake: one incorrect password followed by the correct password from the same normal source.

Observed sequence:

- 1 failed password attempt
- Account: `soc-test`
- Source: `192.168.56.1`
- Successful authentication roughly five seconds later

That pattern was treated as normal user error and should not create a high-priority SOC alert.

Evidence:

- `testing/baseline/03-benign-single-password-mistake.log`

Important Logging Detail

The single bad password generated both:

- `pam_unix(sshd:auth): authentication failure`
- `Failed password for soc-test`

Those entries describe the same failed login. Counting both would inflate the total, so the detector uses the `Failed password` event as the failure record.

Baseline Takeaway

Failed logins are part of normal behavior. The number of failures matters, but so do the source and what happens next in the authentication sequence.

Suspicious Scenario

The suspicious test was generated from Kali Linux at `192.168.56.111` against the `soc-test` account on `ubuntu-soc-lab`.

Activity Generated

- Target account: `soc-test`
- Source IP: `192.168.56.111`
- Destination host: `ubuntu-soc-lab`
- Destination IP: `192.168.56.121`
- Service: SSH
- Failed password attempts: 5
- Total window: 67 seconds
- Successful authentication afterward: No

Why the Sequence Was More Concerning

Compared with the baseline, this activity had several stronger indicators: five failures occurred close together, all targeted one account, the source was outside the established normal source, and the sequence did not end in a successful login.

That combination gave the event more investigative value than a single bad password.

Repeated-Message Compression

The log review also revealed an implementation issue. Ubuntu compressed repeated authentication messages into an entry containing:

`message repeated 2 times`

Five actual password failures therefore appeared as only four matching lines in the log. The detector was updated to interpret the repeated-message count so it would not undercount the activity.

Evidence:

- `testing/true-positive/04-suspicious-five-failures.log`

Data Sources

The primary data source was the Ubuntu authentication log:

`/var/log/auth.log`

The detector relied on SSH authentication records from that file, including:

- `Failed password for <user> from <source IP>`
- `Accepted password for <user> from <source IP>`
- `pam_unix(sshd:session): session opened`
- `pam_unix(sshd:session): session closed`
- `Disconnected from user <user>`
- `message repeated <n> times`

Fields used during correlation included timestamp, username, source IP, source port, destination host, service, authentication result, and failure count.

Data Quality Issue

Repeated-message compression means the number of log lines does not always match the number of real authentication attempts. The implementation explicitly handles that case when counting failed SSH passwords.

Supporting raw logs are stored under `evidence/logs/`.

Detection Design

The detector was written in Python and groups SSH authentication events by username and source IP within a rolling time window.

Version 1

Version 1 alerts when all of these conditions are met:

- SSH password authentication failed.
- The username is the same.
- The source IP is the same.
- At least 5 failed attempts occur.
- The failures fall within 120 seconds.

Rule summary:

`same source + same user + 5 or more failed SSH passwords within 120 seconds`

This version did not consider whether the source was expected for the account or whether the user successfully logged in afterward.

Version 2

Version 2 keeps the same threshold and adds two questions when the threshold is reached:

1. Is the source IP already established as normal for this account?
2. Does the same user successfully authenticate from the same source within 60 seconds after the failure burst?

In the lab, the high-priority alert is suppressed only when both answers are yes. An unusual source or the absence of a successful login keeps the alert active.

For the test account:

- `192.168.56.1` is the known Windows source.
- `192.168.56.111` is treated as unusual.

The detector also understands `message repeated <n> times` entries so compressed log messages are added to the failure count correctly.

Artifacts:

- `detection/scripts/ssh_auth_detection_v1.py`
- `detection/scripts/ssh_auth_detection_v2.py`
- `detection/scripts/ssh_auth_detection.py`
- `detection/rules/detection-logic.md`

Testing Methodology

I tested the detector with scenarios that exercised both the alert path and the no-alert path instead of stopping after the first successful detection.

True Positive

Five failed SSH passwords were generated from Kali `192.168.56.111` against `soc-test` in 67 seconds.

Expected: alert.

True Negative

One failed password was generated from the normal Windows source `192.168.56.1`, followed by a successful login.

Expected: no alert.

False-Positive Challenge

A legitimate user generated five failed passwords from the normal Windows source and then logged in successfully. This deliberately created a pattern that looked suspicious by count and timing but had benign source and sequence context.

Expected after tuning: no high-priority alert.

Boundary Test

The configured count was tested immediately below and at the threshold:

- 4 failures: no alert expected
- 5 failures: alert expected

Regression Test

After tuning Version 2, I reran the original Kali scenario to make sure the false-positive fix had not weakened the tested true-positive behavior.

Test Results

True Positive

- User: soc-test
- Source: 192.168.56.111
- Failures: 5
- Window: 67 seconds
- Successful login afterward: No

Result: **PASS - alert generated.**

Evidence: testing/true-positive/04-true-positive-v2-pass.txt

True Negative

- User: soc-test
- Source: 192.168.56.1
- Failures: 1
- Successful login shortly afterward: Yes

Result: **PASS - no alert generated.**

Evidence: testing/true-negative/01-true-negative-test.txt

False-Positive Test

Version 1 alerted on five legitimate failures from the normal source, followed by a successful login.

Result: **FAIL - false positive.**

Evidence: testing/false-positive/02-false-positive-v1.txt

After adding the Version 2 context checks, the same scenario no longer produced the high-priority alert.

Result: **PASS - expected suppression.**

Evidence: testing/false-positive/03-false-positive-v2-pass.txt

Boundary Test

Four failures stayed below the threshold.

Result: **PASS - no alert.**

Evidence: `testing/boundary-tests/05-boundary-below-threshold-pass.txt`

Five failures met the threshold.

Result: **PASS - alert generated.**

Evidence: `testing/boundary-tests/06-boundary-at-threshold-pass.txt`

Overall Result

Version 2 passed the true-positive, true-negative, false-positive retest, boundary, and regression checks used in the lab.

Tuning History

Version 1 proved that the basic count and time-window logic worked, but the legitimate retry test exposed the rule's biggest weakness. Five bad passwords from the expected Windows source looked identical to the Kali test when only count, account, source, and time were considered.

To address that problem, Version 2 retained the five-in-120 threshold and added source familiarity plus successful-login correlation. For this lab, a high-priority alert is suppressed only when the source is already expected for the user and the same user successfully logs in from that source within 60 seconds after the failures.

The change removed the observed false positive without preventing the Kali test from alerting.

Detailed notes: `engineering-notebook-tuning-history.md`

False Positive Analysis

The clearest false-positive risk in this lab was repeated password entry by a legitimate user.

A real user may type the wrong password several times, retry quickly, and eventually authenticate. On a threshold-only detector, that pattern can be almost indistinguishable from password guessing.

The controlled false-positive scenario produced:

- 5 failed password attempts
- User: `soc-test`
- Source: `192.168.56.1`
- Failure window: 29 seconds
- Successful login immediately afterward

Version 1 alerted on the sequence. Version 2 used the established source and successful follow-up login to suppress the high-priority alert.

Other legitimate situations could still create similar patterns, including administrators mistyping passwords, users connecting from a new system, DHCP address changes, expired credentials in automation, or shared accounts used from multiple approved systems.

These cases are one reason the lab logic should not be copied directly into production without stronger identity, device, and historical context.

Limitations

Static Known-Source Mapping

Version 2 uses a manually defined source mapping. Production logic should learn normal sources from authentication history or trusted device and identity data instead of relying on a hard-coded IP address.

Successful Login Can Still Be Suspicious

A successful login after several failures may mean the attacker eventually found the correct password. Success should only reduce severity when other trusted context supports that conclusion.

Password Authentication Only

The current detector does not evaluate SSH key authentication, MFA events, SSO, Kerberos, or other Linux authentication services.

Low-and-Slow Guessing

An attacker who stays below five failures within 120 seconds can avoid this short-window rule.

Distributed Attempts

The current logic groups by a single source IP. Attempts spread across several sources could remain below the per-source threshold.

Single-Host View

The lab detector reads one Linux host and does not correlate authentication activity across endpoints.

Log Format Dependency

The parser depends on `/var/log/auth.log` formatting. Logging changes, rotation, forwarding, or different message formats could affect results.

No Enterprise-Scale Validation

The lab did not test performance, alert volume, or false-positive rates against production-sized authentication data.

Recommendations

1. Replace the Static Source List

Build source familiarity from historical authentication behavior, device identity, or other trusted identity context.

2. Add Account and Asset Risk

Raise severity when the target is privileged, administrative, service-related, or connected to a sensitive system.

3. Treat Failure-to-Success Sequences Carefully

Do not automatically downgrade an alert because a login eventually succeeded. Success from an unusual source after repeated failures may be more concerning, not less.

4. Add Long-Window Detection

Use additional correlation windows to catch low-and-slow password guessing that stays below the short-term threshold.

5. Look for Distributed Patterns

Correlate one source against many accounts, many sources against one account, and related failures across multiple hosts.

6. Enrich Authentication Context

Where available, add MFA status, device identity, geolocation, source reputation, historical login behavior, asset criticality, account privilege, and recent successful activity.

7. Move Correlation Into a Central Platform

A SIEM or detection platform would make it possible to correlate authentication behavior across many endpoints instead of relying on a local script.

8. Measure Alert Volume Before Raising Severity

Run the rule against representative authentication data, review false positives, and tune both the threshold and severity based on operational impact.

Final Recommendation

Version 2 is a useful lab proof of concept because it demonstrates a clear improvement over the threshold-only approach. Before production use, the logic should be moved into centralized telemetry, supported by dynamic baselining and stronger identity context, and validated against a much larger dataset.